

CHAPTER 09 — PART C

SQLite、Spring Data JPA 與資料持久化

SQLite, Spring Data JPA, and Data Persistence

MiniClinic 社區診所掛號系統

第三週 | 讓 MiniClinic 的資料真的存起來

本週學習目標

Learning Objectives

- 1 說明為什麼需要資料庫 (相較於 Excel 或檔案)
- 2 撰寫基本的 SQL 查詢 (SELECT 、 INSERT 、 UPDATE 、 DELETE 、 WHERE)
- 3 在 Spring Boot 專案中設定 SQLite 連線
- 4 用 JPA 註解把 Java 類別對應到資料表 (@Entity 、 @Id 、 @ManyToOne)
- 5 利用 JpaRepository 實作 CRUD , 不寫一行 SQL
- 6 把 MiniClinic 的資料真正存進資料庫 , 重啟後還在

本週的關鍵轉變

從硬編碼到真實資料庫

Ch09-B 的重大缺陷

資料都是硬編碼在
`DoctorRepository` 裡

- ✗ 每次啟動，資料都一樣
- ✗ POST 新增的掛號，重啟就消失
- ✗ 無法真正新增、修改、刪除
- ✗ 這根本不是一個能運作的系統



Ch09-C 結束時

所有資料存在
`miniclinic.db` 檔案

- ✓ 資料真正儲存在檔案
- ✓ 重啟後資料還在
- ✓ 新增、修改、刪除都生效
- ✓ 真正能運作的系統

為什麼不用 Excel ？

資料庫的價值

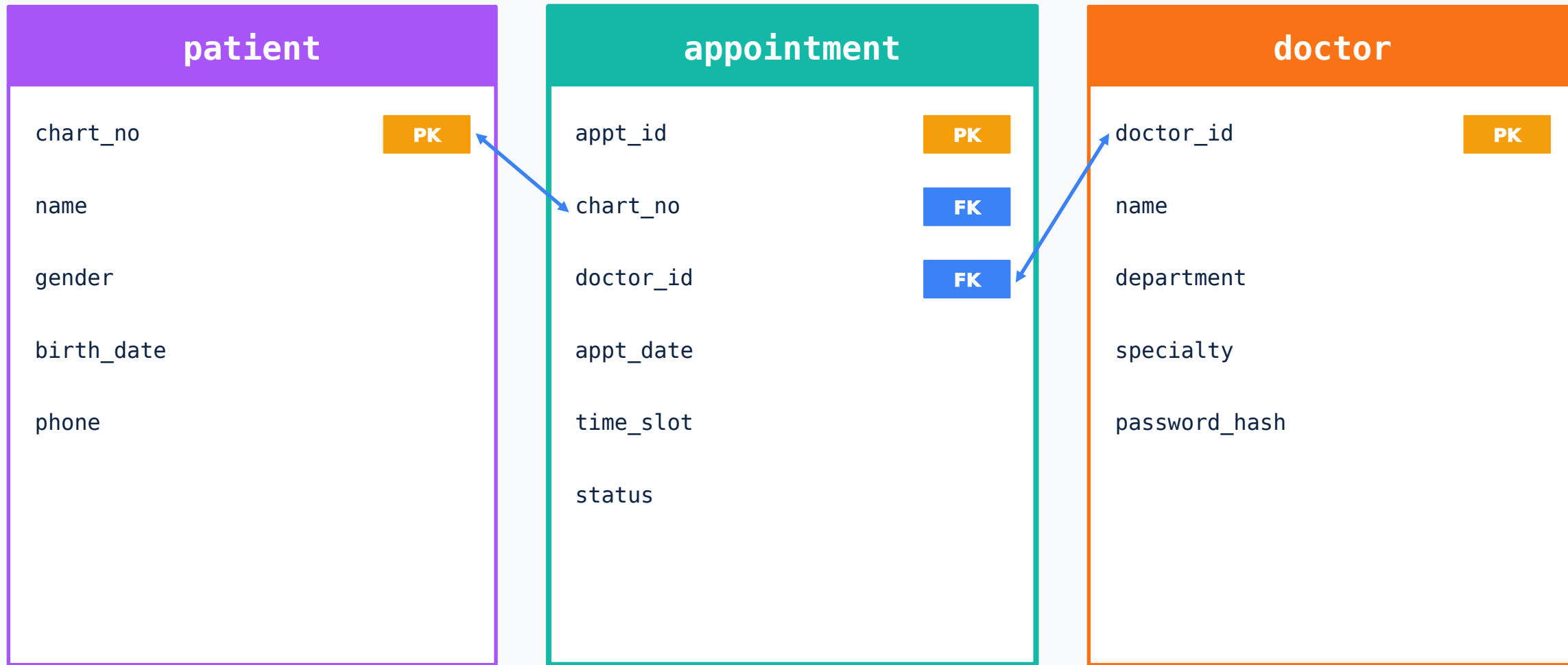
比較項	Excel	資料庫
儲存容量	☐ 約 100 萬列	☒ 數十億筆以上
多人同時修改	☐ 容易衝突、覆蓋	☒ 支援並發、鎖定機制
快速查詢	☐ 資料多時很慢	☒ 有索引，查詢極快
資料完整性	☐ 容易打錯、缺漏	☒ 可設定約束（唯一、非空等）
跨表查詢	☐ 困難、需要公式	☒ JOIN 輕鬆完成
程式化存取	☐ 需要另外寫	☒ 程式直接操作

資料庫核心詞彙

資料表 (Table) · 欄位 (Column) · 記錄 (Row) · **主鍵 (PK)** · **外鍵 (FK)**

MiniClinic 的三張資料表

Patient, Doctor, Appointment



appointment 透過兩個外鍵，記錄「哪位病患預約了哪位醫師」→ 多對一關係

SQL 基礎速覽

四個核心動作：SELECT / INSERT / UPDATE / DELETE

SELECT

查詢



```
SELECT * FROM doctor  
WHERE department = '內科';
```

INSERT

新增



```
INSERT INTO doctor  
  (doctor_id, name, ...)  
VALUES ('D006', '吳大明', ...);
```

UPDATE

修改



```
UPDATE doctor  
SET specialty = '新的專長'  
WHERE doctor_id = 'D002';
```

DELETE

刪除



```
DELETE FROM doctor  
WHERE doctor_id = 'D005';
```

UPDATE / DELETE 別忘了 WHERE ！省略會把整張表清掉

為什麼選 SQLite ?

零配置、單一檔案、真實世界用途極廣

01

單一檔案，零配置

不用安裝伺服器、不用帳密，所有資料在一個 .db 檔案

02

嵌入式

不是獨立程式，而是一個函式庫；應用直接讀寫檔案

03

標準 SQL

和 PostgreSQL、MySQL 語法絕大多數相容

你可能沒想到，SQLite 每天都在你身邊

iPhone / Android

每個 App 的本地資料庫

Firefox / Chrome

歷史紀錄、書籤、Cookie

WhatsApp / Line

聊天紀錄儲存

你用過的軟體

約一半背後都有 SQLite

加入依賴與設定

pom.xml 與 application.properties

pom.xml 新增三個依賴



```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.45.0.0</version>
</dependency>

<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-community-dialects</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

application.properties 設定



```
# SQLite 資料庫檔案位置
spring.datasource.url=jdbc:sqlite:miniclinic.db
spring.datasource.driver-class-name=
  org.sqlite.JDBC

# Hibernate SQLite 方言
spring.jpa.database-platform=
  org.hibernate.community.dialect.SQLiteDialect

# 自動建表 (開發用)
spring.jpa.hibernate.ddl-auto=update

# 顯示執行的 SQL
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```


spring.jpa.hibernate.ddl-auto 選項

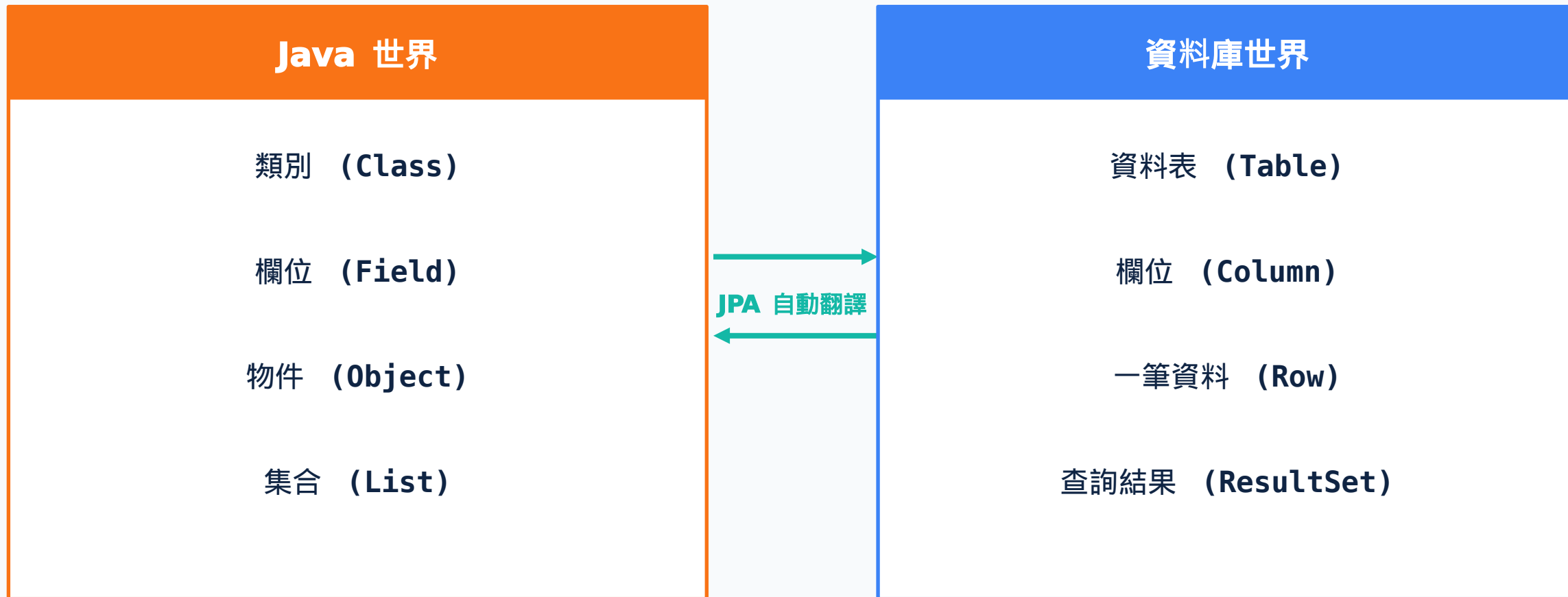
控制啟動時的資料表行為

值	作用	適用情境
none	什麼都不做	正式環境
validate	檢查 Entity 是否對應資料表	正式環境
update	資料表不存在就建立，缺欄位就加	開發環境（我們用這個）
create	每次啟動先刪掉再重建	測試
create-drop	啟動建立，關閉時刪除	測試

提示：Ch09-E 部署到雲端時會改成 `validate` 或 `none`，搭配 Flyway 等工具做版本管理

JPA / ORM 核心觀念

用物件導向的方式操作資料庫



你寫 Java , JPA 自動產生 SQL 。 Hibernate 是 JPA 規範最流行的實作

Doctor Entity 加上 JPA 註解

打開 `Doctor.java` , 加上 JPA 註解。從 POJO 變成可持久化的類別 (POJO: Plain Old Java Object , 普通 Java 對象)



```
import jakarta.persistence.*;

@Entity
@Table(name = "doctor")
public class Doctor {

    @Id
    @Column(name = "doctor_id", length = 10)
    private String doctorId;

    @Column(name = "name", length = 50, nullable = false)
    private String name;

    @Column(name = "department", length = 20, nullable = false)
    private String department;

    @Column(name = "specialty", length = 100)
    private String specialty;

    public Doctor() {} // JPA 需要無參建構子

    public Doctor(String doctorId, String name, ...) { ... }
    // Getters & Setters 省略 ...
}
```

四個關鍵註解

@Entity

對應一張資料表

@Table

指定資料表名稱

@Id

標記主鍵欄位

@Column

指定欄位細節

Patient Entity (請課後對照講義自學)

和 Doctor 結構相似，可舉一反三



```
import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "patient")
public class Patient {

    @Id
    @Column(name = "chart_no", length = 20)
    private String chartNo;

    @Column(name = "name", length = 50, nullable = false)
    private String name;

    @Column(name = "gender", length = 10)
    private String gender;

    @Column(name = "birth_date")
    private LocalDate birthDate;

    @Column(name = "phone", length = 20)
    private String phone;

    public Patient() {}
    // 有參建構子、Getters、Setters 省略 ...
}
```

和 Doctor 的差別

主鍵是 **chartNo**

病歷號格式 TEST00001 (虛構教學格式)

LocalDate 型別

Java 8+ 日期型別，對應 SQL 的 DATE 欄位

結構幾乎相同

其他註解用法完全一樣，課後對照講義自行完成

Appointment Entity 與 @ManyToOne

表達資料表之間的關聯



```
@Entity
@Table(name = "appointment")
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "appt_id")
    private Long apptId;

    @ManyToOne
    @JoinColumn(name = "chart_no", nullable = false)
    private Patient patient;

    @ManyToOne
    @JoinColumn(name = "doctor_id", nullable = false)
    private Doctor doctor;

    @Column(name = "appt_date", nullable = false)
    private LocalDate apptDate;

    @Column(name = "time_slot", length = 20)
    private String timeSlot;

    @Column(name = "status", length = 20)
    private String status = "BOOKED";
    // 建構子、Getters、Setters 省略...
}
```

三個新註解

@GeneratedValue

主鍵由資料庫自動產生 (流水號)

@ManyToOne

表達「多對一」關係，JPA 自動 JOIN

@JoinColumn

指定外鍵欄位名稱

為什麼本課程只用單向 @ManyToOne ?

雙向關聯的坑太多，本課程避開

cascade 陷阱

刪掉 Patient 要不要連帶刪掉他的 Appointment ?

後果：
刪一筆病患→誤刪一堆掛號

fetch 策略

每次讀 Patient 就要撈出所有 Appointment ?

後果：
查一位病患 → 慢到無法接受

JSON 無限遞迴

序列化時 Patient → Appointments → Patient...

後果：
StackOverflowError 崩潰

本課程的做法

只用單向 @ManyToOne : Appointment 指向 Patient / Doctor , 反向不做。
要查「某病患的所有掛號」時：
`appointmentRepo.findByPatient(patient)` ← 功能完全不受影響

Spring Data JPA 的魔法

三行程式碼，取代整個 Repository

之前 (Ch09-B)



```
@Component
public class DoctorRepository {

    private static final List<Doctor>
        DOCTORS = Arrays.asList(...);

    public List<Doctor> findAll() {
        return DOCTORS;
    }

    public Optional<Doctor> findById(
        String doctorId) {
        return DOCTORS.stream()
            .filter(...)
            .findFirst();
    }

    public List<Doctor> findByDepartment(
        String department) {
        return DOCTORS.stream()
            .filter(...)
            .toList();
    }
}
```

現在 (Ch09-C)



```
public interface DoctorRepository
    extends JpaRepository<Doctor, String> {

    List<Doctor> findByDepartment(
        String department);
}
```



不用寫實作！

JpaRepository 提供開箱方法：

- findAll(), findById(id)
- save(entity)
- deleteById(id)
- count(), existsById(id)

加上命名查詢，自動產生 SQL。

方法命名查詢 (Query Derivation)

Spring 看方法名，自動產生 SQL

方法名	產生的 SQL
<code>findByName(String name)</code>	<code>WHERE name = ?</code>
<code>findByNameAndDepartment(...)</code>	<code>WHERE name = ? AND department = ?</code>
<code>findByDepartmentOrderByName()</code>	<code>... ORDER BY name</code>
<code>countByDepartment(...)</code>	<code>SELECT COUNT(*) WHERE department = ?</code>
<code>findByApptDateBetween(from, to)</code>	<code>WHERE appt_date BETWEEN ? AND ?</code>
<code>existsByDoctorId(id)</code>	<code>SELECT COUNT > 0</code>

只要方法名符合規則，**Spring** 就會自動產生實作

欄位名必須和 Entity 的欄位一致（駝峰式）。你不用寫任何 SQL。

Controller 一個字都不用改！

上週的「介面一致命名」設計兌現了



```
@RestController
public class DoctorApiController {

    @Autowired
    private DoctorRepository doctorRepo;

    @GetMapping("/api/doctors")
    public List<Doctor> getDoctors(
        @RequestParam(required = false) String department) {
        if (department == null || department.isBlank()) {
            return doctorRepo.findAll();
        }
        return doctorRepo.findByDepartment(department);
    }

    @GetMapping("/api/doctors/{doctorId}")
    public ResponseEntity<Doctor> getDoctor(@PathVariable String doctorId) {
        return doctorRepo.findById(doctorId)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

因為新舊 **DoctorRepository** 的方法簽章完全一致：**findAll()**, **findById()**, **findByDepartment()**

data.sql 初始化資料

每次啟動自動插入基本資料

src/main/resources/data.sql



```
-- 初始醫師 ( 5 位 )
INSERT INTO doctor VALUES
('D001', '陳志明醫師', '家醫科', '一般內科、慢性病管理'),
('D002', '林佩君醫師', '內科', '心臟血管、高血壓'),
('D003', '王建華醫師', '復健科', '運動傷害、脊椎復健'),
('D004', '李美玲醫師', '小兒科', '兒童感冒、疫苗接種'),
('D005', '張雅筑醫師', '身心科', '焦慮、失眠、情緒調適');

-- 初始病患 ( 3 位虛構病患 )
INSERT INTO patient VALUES
('TEST00001', '測試病患甲', '男', '1985-03-15', '0912-345-678'),
('TEST00002', '王小明', '男', '1990-07-22', '0923-456-789'),
('TEST00003', '李小華', '女', '1988-11-30', '0934-567-890');

-- 初始掛號 ( 3 筆 )
INSERT INTO appointment
(chart_no, doctor_id, appt_date, time_slot, status) VALUES
('TEST00001', 'D001', '2026-05-01', 'AM', 'BOOKED'),
('TEST00002', 'D002', '2026-05-01', 'AM', 'BOOKED'),
('TEST00003', 'D003', '2026-05-02', 'PM', 'BOOKED');
```

application.properties



```
spring.jpa
    .defer-datasource
    -initialization=true

spring.sql.init
    .mode=always
```

啟動流程

1. 建立 / 開啟 .db 檔
2. JPA 建資料表
3. 執行 data.sql
4. 啟動完成

Doctor CRUD API 完整實作

示範 Create / Update / Delete 三個新端點



```
// Create
@PostMapping("/api/doctors")
public ResponseEntity<Doctor> createDoctor(@RequestBody Doctor doctor) {
    Doctor saved = doctorRepo.save(doctor);
    return ResponseEntity.status(201).body(saved);
}

// Update
@PutMapping("/api/doctors/{doctorId}")
public ResponseEntity<Doctor> updateDoctor(
    @PathVariable String doctorId,
    @RequestBody Doctor updated) {
    return doctorRepo.findById(doctorId)
        .map(existing -> {
            existing.setName(updated.getName());
            existing.setDepartment(updated.getDepartment());
            existing.setSpecialty(updated.getSpecialty());
            return ResponseEntity.ok(doctorRepo.save(existing));
        })
        .orElse(ResponseEntity.notFound().build());
}

// Delete
@DeleteMapping("/api/doctors/{doctorId}")
public ResponseEntity<Void> deleteDoctor(@PathVariable String id) {
    if (!doctorRepo.existsById(id)) return ResponseEntity.notFound().build();
    doctorRepo.deleteById(id);
    return ResponseEntity.noContent().build(); // 204
}
```

REST 狀態碼選擇

不同情境回不同狀態碼

情境	正確狀態碼	常見錯誤
查詢成功	200 OK	
新增成功	201 Created	200 勉強可用，不精準
刪除成功（無回應內容）	204 No Content	
找不到資源	404 Not Found	200 + 錯誤訊息（壞做法）
輸入格式錯誤	400 Bad Request	500
未登入（下週）	401 Unauthorized	200
已登入但無權限	403 Forbidden	401

作業三：讓 MiniClinic 的資料真的存起來

Homework Ch09-C

Part A：基本要求（必做）

- ☐ 依賴與 application.properties 設定正確
- ☐ 三個 Entity：Doctor / Patient / Appointment
- ☐ Appointment 使用 @ManyToOne 關聯
- ☐ 三個 Repository 繼承 JpaRepository
- ☐ data.sql 含 5 醫師、3 病患、3 掛號
- ☐ POST/PUT/DELETE /api/doctors 全實作
- ☐ 掛號表單真的存入資料庫
- ☐ GET /api/patients、/api/appointments 實作
- ☐ api-tests.http 涵蓋所有新 API

Part B：掛號統計頁（必做）

新增頁面 **GET /stats** 顯示：

- 醫師總數、病患總數、掛號總數
- 依科別分組，列出每科的掛號數

新增三個 **API**：

- GET /api/appointments/count
- GET /api/appointments?date=YYYY-MM-DD
- GET /api/appointments?doctorId=D001

提示

AppointmentRepository 加入
findByApptDate、findByDoctor、count() 等命名方法即可。

本週作業繳交

- 延續 Ch09-B 的 miniclinic 專案繼續開發
- 壓縮為 zip 檔，檔名：學號_姓名_Ch09C.zip
- 上傳至教學平台
- 記得刪除 target/ 與 miniclinic.db （讓助教看到乾淨初始狀態）

下週預告：Ch09-D

Session、驗證與醫療資料安全

- HTTP 無狀態、Cookie 與 Session 的差別
- 手寫醫師登入流程（不用 Spring Security）
- Interceptor 保護需登入的路由、BCrypt 密碼雜湊
- Bean Validation（@NotBlank、@Pattern、@Valid）
- SQL Injection、XSS、CSRF 與醫療資料倫理